Sprint 1 - Planejamento (Fundação + Peças/Estoque)

Projeto: Sistema de Gestão — Oficina Hudson Car (Charlles Hudson) · Duração: 2 semanas (10 dias úteis) · Time: 5 desenvolvedores · Stack: Next.js + TypeScript + PostgreSQL (Prisma) + Vitest/RTL/Playwright

1. Objetivo da Sprint (Sprint Goal)

Ter a fundação do sistema no ar (auth + RBAC + design base + CI) e o módulo de maior dor do cliente funcionando de ponta a ponta: cadastro e busca de peças/fornecedores, controle de estoque com alertas de compra e "preço do dia".

Ao final da sprint, um usuário operacional (Vitor) deve conseguir:

- Fazer login e navegar num layout consistente e simples.
- Cadastrar/editar peças e fornecedores.
- Buscar peça por nome, código e chassi compatível.
- Ver o estoque atualizado, dar entrada/baixa e receber alertas de compra quando abaixo do mínimo.
- Atualizar manualmente o preço do dia de cada peça.

E o admin (Charlles) deve ver tudo isso sem que dados financeiros vazem para os demais papéis (regra de ouro nº 2).

Fora do escopo desta sprint: orçamento, ordem de serviço, relatórios, fotos de entrada/saída, migração real de dados (apenas spike) e site institucional.

2. Princípios que guiam a sprint

- Usabilidade acima de tudo, telas simples, sem jargão. Toda UI passa pelo teste: "o Charlles conseguiria usar sozinho?".
- Financeiro é sigiloso, nenhuma rota/campo financeiro fora do papel admin.
- Preço da peça = preço do dia, campo editável manualmente, sem média nem trava, com auditoria de quem/quando.
- Concorrência não pode quebrar dados, o modelo nasce com controle de versão otimista (version) para preparar a edição simultânea.
- Idioma do domínio em português, peca, estoque, fornecedor, veiculo, cliente, usuario, papel.

3. Time e responsabilidades (5 devs)

Dev	Papel na sprint	Foco principal	Agentes de apoio
D1	Tech Lead / Frontend	Scaffold do projeto, Prisma+Postgres, migrations, CI, config de ambiente, code review	backend-architect, code-reviewer
D2	Backend — wPeças/Estoque	API de peças, fornecedores, estoque, movimentações e preço do dia	backend-architect, api-endpoint
D3	Backend — Auth & RBAC	Autenticação, papéis, middleware de autorização e guarda de rotas financeiras	backend-architect, security-auditor
D4	Frontend — Design System & Fundação	Layout, navegação, design system, tela de login e telas de fornecedores	frontend-architect, react-component
D5	Backend — Peças/Estoque & Auth	Busca de peças (inclui chassi), listagem, estoque, alertas e preço do dia	frontend-architect, test-engineer

Testes são responsabilidade de todos: D1 configura o ferramental e cada dev entrega sua fatia com testes; D5 lidera os e2e do fluxo de peças/estoque.

4. Decisões técnicas

Área	Decisão	Observação
Framework	React.js + Node.js	Front + API (route handlers / server actions) no mesmo repo
Linguagem	TypeScript	noUncheckedIndexedAccess ligado
ORM / DB	Prisma + PostgreSQL	Postgres local via Docker Compose
Auth	Auth.js (NextAuth) ou Lucia	Sessão + RBAC por papel; senha com hash (argon2/bcrypt)
UI	Tailwind + shadcn/ui	Base acessível, acelera um design system simples
Testes	Vitest + RTL + Playwright	Cobertura nas regras de negócio
Lint / format	ESLint + Prettier	Gate no CI

5. Modelo de dados e RBAC

Entidades (nascem nesta sprint)

Todas com id, createdAt, updatedAt e version (int, controle otimista).

- Usuario — nome, email, senhaHash, papel (ADMIN | GESTOR | OPERACIONAL), ativo.
- Fornecedor — nome, doc, telefone, email, observacao.
- Peca — nome, codigo, categoria, subcategoria, fornecedorId, unidade, precoDia, precoDiaAtualizadoEm/Por, quantidadeAtual, estoqueMinimo.
- ChassiCompativel — pecaId, chassi, modeloVeiculo (viabiliza a busca por chassi).
- MovimentacaoEstoque — pecaId, tipo (ENTRADA | SAIDA | AJUSTE), quantidade, motivo, usuarioId, data.

Regras de negócio embutidas

- Toda alteração de quantidadeAtual passa por uma MovimentacaoEstoque (rastreabilidade).
- precoDia nunca é calculado nem mediado — só sobrescrito manualmente, com auditoria de autor e data.
- Campos financeiros de fato não entram nesta sprint; quando entrarem, ficam em entidade/rota separada, protegida por RBAC ADMIN.

Matriz de permissões (RBAC)

Recurso	ADMIN (Charlles)	GESTOR (irmão)	OPERACIONAL (Vitor)
Peças (CRUD)	Sim	Sim	Sim
Fornecedores (CRUD)	Sim	Sim	Sim
Estoque / movimentações	Sim	Sim	Sim
Preço do dia (editar)	Sim	Sim	Sim
Qualquer dado financeiro	Sim	Não	Não
Gestão de usuários/papéis	Sim	Não	Não

OBS: Nesta sprint não há tela financeira ainda.

Épico A — Fundação da Plataforma (D1, apoio D3)

ID	História	Critérios de aceite
A1	Scaffold Next.js + TS + estrutura por domínio	pnpm dev sobe; estrutura documentada; strict mode
A2	Postgres (Docker) + Prisma + migration inicial	docker compose sobe o DB; migrate cria as tabelas
A3	Seed de dados (usuários dos 3 papéis, fornecedores, peças)	pnpm seed popula uma base navegável
A4	CI (GitHub Actions): lint + typecheck + test	PR bloqueia merge se falhar
A5	Convenções: ESLint/Prettier, README, padrão de erro da API	Doc no repo; erro { code, message }

Épico B — Autenticação & RBAC (D3, apoio D1)

ID	História	Critérios de aceite
B1	Login/logout com sessão + senha com hash	Usuário inválido rejeitado; sessão persiste; senha nunca em texto plano
B2	Papéis + middleware de autorização	Rota protegida por papel; 403 para papel sem acesso
B3	Guarda de rotas/campos financeiros (regra nº 2)	Teste prova que GESTOR/OPERACIONAL não acessam financeiro
B4	Auditoria mínima (quem alterou preço/estoque)	Campos de autor preenchidos e testados

Épico C — Peças & Fornecedores / Backend (D2)

ID	História	Critérios de aceite
C1	API CRUD Fornecedor (Zod + testes)	CRUD com validação; suporta 20-25 fornecedores
C2	API CRUD Peça (categoria/subcategoria, unidade)	CRUD completo com validação e testes
C3	Busca de peça por nome, código e chassi	Filtros combináveis; busca por chassi correta; testes
C4	Atualização de preço do dia com auditoria	Endpoint dedicado; grava autor/data; sem cálculo automático

Épico D — Estoque & Alertas (D2 + D5)

ID	História	Critérios de aceite
D1x	API de movimentação (entrada/saída/ajuste)	Toda mudança de saldo gera movimentação; testes
D2x	Alertas de compra (abaixo do mínimo)	Lista de peças em alerta; regra testada (óleo, presilhas)

Épico E — Frontend Fundação + Peças/Estoque (D4 + D5)

ID	História	Critérios de aceite
E1	Layout + navegação + design system	Componentes base acessíveis (botão, input, tabela, modal, toast)
E2	Tela de login	Formulário simples, erros claros, responsivo
E3	Telas de Fornecedores (lista + form)	Lista, criar, editar; validação e feedback
E4	Busca/listagem de Peças (nome, código, chassi)	Busca rápida; chassi em destaque
E5	Tela de Peça (detalhe + preço do dia)	Editar dados e preço; mostra atualizado por/em

E6	Estoque + movimentações + painel de alertas	Ver saldo, entrada/baixa, alertas na home
E7	Esconder financeiro na UI conforme papel	Componentes financeiros não renderizam p/ GESTOR/OPERACIONAL

6. Cronograma (2 semanas)

Semana 1 — Fundação e contratos

Dia	Foco	Entregáveis-chave
Dia 1	Kickoff + setup	A1 iniciado; decisões técnicas confirmadas; repo com scaffold
Dia 2	Fundação DB/CI + contratos	A2 (schema+migration), A4/A5; contratos Zod acordados
Dia 3	Auth + domínio + design system	B1; C1 (fornecedor); E1 (design system)
Dia 4	RBAC + peças + UI fornecedor	B2; C2 (peça); E2/E3
Dia 5	RBAC financeiro + busca peça	B3/B4; C3 começa; E4 começa

Semana 2 — Peças/Estoque de ponta a ponta

Dia	Foco	Entregáveis-chave
Dia 6	Busca + preço do dia	C3 finalizada; C4 (preço do dia); E4/E5
Dia 7	Estoque	D1x (movimentações); E6 começa
Dia 8	Alertas + integração	D2x (alertas); E6 (painel de alertas); E7
Dia 9	Hardening	e2e do fluxo completo; correção de bugs; revisão de segurança
Dia 10	Fechamento	Regressão, polish de usabilidade, demo + review + retro